

mahg_bigdata_rdd

Resilient Distributed Datasets in Apache Spark: Architecture, Execution Semantics, and FinOps-Aware Resource Estimation

Manuel Hernandez Giuliani

Independent Researcher and Instructor

manuelhernandezgiuliani.com

Publication Date: 2026-04-19

Publication No.: MAHG-BD-PAPER-RDD-20260419-01

Abstract

Apache Spark remains one of the most influential distributed computing engines for large-scale data processing, yet its foundational concepts are often taught too quickly or reduced to operational recipes. This article revisits Apache Spark from the perspective of resilient distributed datasets (RDDs), execution flow, scheduling, and practical resource estimation. Drawing primarily on the official Spark 4.1.1 documentation and the seminal RDD paper, the discussion explains how the driver, cluster manager, executors, partitions, tasks, stages, shuffles, persistence levels, and shared variables work together as one coherent runtime model. It also clarifies an important pedagogical issue: Spark does not prescribe one universal formula for sizing executors or memory, so classroom estimation must be presented as an engineering heuristic validated through the Spark UI and workload observation rather than as fixed doctrine. Using ManOxCo as the exclusive business case, the paper shows how Spark concepts support life-critical analytics, periodic reconciliation, and cost-aware scaling decisions in a modern Big Data platform (Apache Spark, n.d.-a, n.d.-f; Zaharia et al., 2012).

Keywords: Apache Spark, resilient distributed datasets, RDD, distributed computing, partitions, Spark scheduling, dynamic allocation, FinOps

1. Introduction

Apache Spark is often introduced as a “fast Big Data engine,” but such a description is incomplete for academic and architectural purposes. Spark is better understood as a distributed execution model built around partitions, fault tolerance, lazy evaluation, and

coordinated resource management. Its performance comes not only from memory usage or parallelism, but from the way it structures work across drivers, executors, tasks, stages, and shared cluster resources (Apache Spark, n.d.-a, n.d.-e).

The concept of the resilient distributed dataset remains especially important in that explanation. Even when practitioners increasingly rely on higher-level abstractions such as DataFrames and SQL, the RDD model still provides the clearest conceptual path for understanding how Spark reasons about lineage, transformations, actions, persistence, and recovery from node failure (Apache Spark, n.d.-e; Zaharia et al., 2012). For students, this makes RDDs more than a legacy API: they are the intellectual foundation for understanding distributed execution.

This article therefore treats Spark as a systems topic, not merely as a coding topic. It begins with the RDD abstraction, then explains the runtime architecture and execution flow, and finally reframes resource sizing, autoscaling, and FinOps as workload-dependent design choices. The teaching emphasis is deliberately practical: in ManOxCo, a Spark workload is not interesting because it processes data at scale alone, but because it can help transform oxygen production, delivery, and hospital-consumption signals into timely and trustworthy action.

“We don’t start with data or technology. We start with the business. Data follows, and cloud becomes irrelevant when you master its lingua franca: Apache Spark.”

Manuel Hernandez Giuliani

2. Objectives

This article pursues five principal objectives. First, it clarifies the formal meaning of an RDD and explains why fault tolerance, partitioning, and lazy transformations remain central to Spark literacy. Second, it describes Spark’s runtime architecture by distinguishing the roles of `SparkSession`, `SparkContext`, the driver, cluster manager, executors, worker nodes, and the submission process. Third, it explains how actions trigger jobs, how jobs are divided into stages and tasks, and why shuffles become decisive performance boundaries. Fourth, it replaces simplistic executor-sizing formulas with a more accurate resource-estimation methodology grounded in the official Spark documentation, hardware guidance, and runtime observation. Fifth, it connects these concepts to the ManOxCo case so that Spark is understood as an enabler of business-critical data engineering rather than as an isolated technical artifact.

3. Main Discussion

3.1. The RDD as Spark's Foundational Abstraction

The official Spark documentation defines an RDD as a fault-tolerant collection of elements that can be operated on in parallel across a cluster (Apache Spark, n.d.-e). This definition is compact, but conceptually dense. “Distributed” means the data is partitioned across nodes rather than stored as one local object. “Fault-tolerant” means the system can recover lost partitions after node failure. “Operated on in parallel” means the workload is expressed as transformations and actions that Spark can schedule across multiple executors.

Zaharia et al. (2012) provide the deeper architectural significance of that abstraction. In their original formulation, RDDs allow in-memory computations on large clusters while preserving fault tolerance through coarse-grained transformations instead of fine-grained shared-memory mutation. This matters because Spark is not designed around arbitrary shared-state editing; it is designed around repeatable lineage and structured recomputation. As a result, immutability is not merely a programming convenience. It is one of the reasons Spark can recover efficiently and reason about execution across distributed machines.

The RDD abstraction also clarifies why Spark often feels different from conventional scripting. An RDD is not just a container of records. It is a distributed plan for how data can be derived, partitioned, reused, and recomputed. For teaching purposes, this is one of the most important conceptual upgrades students must make when moving from local programming to cluster computing.

3.2. Spark Runtime Architecture

Spark applications run as coordinated processes on a cluster. In modern Spark, the practical application entry point is often `SparkSession`, while `SparkContext` remains the foundational cluster-facing context. The official SQL getting-started guide describes `SparkSession` as the entry point into Spark functionality, while the cluster overview explains how the application context connects to a cluster manager, acquires executors on cluster nodes, sends application code, and dispatches tasks for execution (Apache Spark, n.d.-a, n.d.-g).

For teaching purposes, students should understand both names together rather than as competing alternatives. `SparkSession` is the modern interface through which many applications create `DataFrames`, run SQL, and access structured APIs, whereas `SparkContext`

remains the lower-level coordination object that represents the application's relationship to cluster resources (Apache Spark, n.d.-e, n.d.-g).

This architecture can be interpreted through four practical roles. The driver is the control plane for the application: it holds the application logic, coordinates scheduling, and receives final results. The cluster manager is the resource broker: Spark can operate with its standalone cluster manager, YARN, or Kubernetes, each of which allocates resources differently (Apache Spark, n.d.-a). Executors are the worker-side processes that perform computations and store data for the application. Worker nodes are the underlying machines that host executors and provide CPU, memory, disk, and network capacity.

One important teaching point is that Spark applications are isolated at the executor level. According to the scheduling documentation, each Spark application gets an independent set of executor processes that run tasks and store data for that application (Apache Spark, n.d.-d). This means multi-application coordination is not achieved by one common memory pool across applications, but by cluster-level resource policies and scheduling decisions.

3.3. Execution Flow: From Actions to Jobs, Stages, and Tasks

Spark's execution flow becomes easier to understand when one begins with actions rather than with static diagrams. The RDD guide explains that transformations create a new dataset from an existing one, whereas actions return a value to the driver after running a computation on the dataset (Apache Spark, n.d.-e). Transformations are lazy: they record intent without immediately computing results. Execution begins when an action requires a result.

Once an action is triggered, Spark materializes work as jobs, stages, and tasks. The scheduling guide defines a "job" as a Spark action and the tasks needed to evaluate it (Apache Spark, n.d.-d). The RDD guide further states that actions are executed through a set of stages separated by shuffle operations (Apache Spark, n.d.-e). In practice, a stage groups tasks that can run without repartitioning data across the cluster, whereas a shuffle introduces a boundary where data must be redistributed.

Tasks are the smallest execution units dispatched to executors. The RDD guide notes that Spark will run one task for each partition in the relevant dataset or parallel collection (Apache Spark, n.d.-e). This is why partitions become the practical unit of parallelism: too few partitions underutilize the cluster, whereas too many tiny partitions may increase scheduling overhead.

The shuffle is especially important in academic explanation because it marks the transition from local partition-level processing to network-intensive redistribution. Spark describes the shuffle as its mechanism for regrouping data across partitions, often by copying data across executors and machines, which makes it both complex and costly (Apache Spark, n.d.-e). Shuffles therefore separate conceptually simple pipelines from operationally expensive ones.

3.4. Core RDD Concepts That Students Must Master

Four RDD concepts are foundational for accurate reasoning about Spark. The first is immutability. Every transformation produces a new RDD rather than editing the original one, which supports predictable lineage and recomputation (Apache Spark, n.d.-e; Zaharia et al., 2012). The second is laziness. Spark does not compute transformed datasets immediately; instead, it waits until an action requires output. This allows Spark to optimize execution and avoid unnecessary materialization (Apache Spark, n.d.-e).

The third concept is persistence. By default, Spark may recompute transformed RDDs whenever a later action needs them. The RDD guide explains that `persist()` or `cache()` can keep the elements available on the cluster for faster reuse, and it documents multiple storage levels such as `MEMORY_ONLY`, `MEMORY_AND_DISK`, and serialized alternatives (Apache Spark, n.d.-e, n.d.-f). Persistence is therefore a strategic decision: it trades memory and storage resources for reduced recomputation.

The fourth concept is shared execution support. Spark ships variables used in parallel functions to tasks by default, but it also provides broadcast variables and accumulators as explicit shared abstractions (Apache Spark, n.d.-e). Broadcast variables help distribute read-mostly data efficiently, while accumulators support aggregate updates such as counters. These are not peripheral features; they are part of how Spark manages distributed coordination without reverting to unrestricted shared memory.

For performance analysis, students should also understand that some operations are key-value specific and often shuffle-intensive. Grouping, aggregation, and `reduceByKey`-style workloads can become memory- and network-heavy because Spark must reorganize data by key across partitions (Apache Spark, n.d.-e). In other words, the conceptual beauty of a transformation chain does not exempt it from physical execution costs.

3.5. Resource Estimation as an Engineering Heuristic, Not a Universal Formula

The original draft presented resource sizing as if Spark prescribed a single deterministic formula for partitions, executors, and memory. The official documentation does not support that interpretation. Spark does provide guidance, but that guidance is conditional and workload-sensitive. For example, the RDD programming guide notes that, for parallelized collections, a typical target is two to four partitions per CPU in the cluster, while still allowing Spark to set partitions automatically in many cases (Apache Spark, n.d.-e). This is advice, not a universal law for every data source and storage layout.

The hardware provisioning guide is similarly pragmatic. It recommends at least 8 to 16 CPU cores per machine in many cases, advises leaving part of machine memory to the operating system and buffer cache, and explicitly suggests using the Spark monitoring UI to observe the memory footprint of sampled RDDs instead of guessing blindly (Apache Spark, n.d.-c). It also recommends allocating at most about 75% of machine memory to Spark, with the remainder reserved for the operating system and related services (Apache Spark, n.d.-c).

From a teaching standpoint, however, students still need a reproducible manual method before they can appreciate automation. For that reason, this article restores a classroom sizing workflow as a pedagogical heuristic. The key is to be explicit that the workflow is instructional, not canonical Spark doctrine. It teaches students what technical variables matter and why each variable changes the final architecture.

3.5.1. Technical Inputs Students Must Identify Before Calculating

Before any arithmetic, students must identify the technical assumptions behind the workload.

First, they must normalize the data volume into a consistent unit, typically mebibytes or gibibytes. If a file is 10 GB, and the class is using binary units for storage estimation, the teaching conversion is $10 \times 1024 = 10,240$ MiB.

Second, they must define a target partition size. In Spark, partitions are the unit of parallelism, but Spark does not impose one mandatory partition size for every workload. For introductory file-based examples, a 128 MiB target partition is a reasonable teaching assumption because it aligns with a common HDFS-style split size and keeps the arithmetic clear. The important lesson is that the target partition size is an architectural assumption, not a universal law.

Third, students must define the desired concurrency model. If a dataset produces 80 partitions, Spark does not require 80 cores to run the job. Spark can run those tasks in waves. The classroom examples below intentionally use a one-wave model in which all target tasks can start immediately. This makes the math visible and teaches the relationship between partitions and task slots. In real life, architects often trade completion time for lower cost by accepting multiple waves.

Fourth, students must choose a number of cores per executor. For introductory calculations, 2 cores per executor is a simple and conservative assumption because it keeps the arithmetic transparent and avoids pretending that large executor sizes are always beneficial. In practice, this value depends on garbage collection, shuffle intensity, and cluster topology.

Fifth, students must estimate working memory. Here the original draft contained a useful intuition but presented it too strongly. The formula

$$\text{memory per executor} = \text{partition size} \times \text{memory amplification factor} \times \text{cores per executor}$$

can still be used in class if its meaning is explained correctly. The memory amplification factor is not a Spark constant. It is a classroom proxy for decompression, deserialized objects, shuffle buffers, temporary rows, and similar runtime expansion. In these introductory examples, the factor is 4. That means each active core is assumed to need roughly four times its partition size as working memory.

Sixth, students must add executor overhead. In the classroom model, a 20% overhead is added to cover executor-side runtime needs beyond the base working estimate. The calculation becomes:

$$\text{final executor memory} = \text{base executor memory} / (1 - 0.20)$$

which is equivalent to dividing by 0.8.

3.5.2. Classroom Estimation Workflow

Using those assumptions, a student can estimate resources through the following sequence:

1. Convert input data to MiB.
2. Estimate partitions with
$$\text{partitions} = \text{ceil}(\text{data size in MiB} / \text{target partition size in MiB}).$$

3. Estimate active tasks according to the desired concurrency model.

For one simultaneous batch execution, `active tasks = partitions`.

For two simultaneous executions, `active tasks = partitions x 2`.

For mixed workloads, add the active batch and streaming partitions together.

4. Choose executor cores. In the worked examples below, 2 cores per executor are used.

5. Estimate executors with

`executors = ceil(active tasks / cores per executor)`.

6. Estimate base executor memory with

`base executor memory = partition size x amplification factor x cores per executor`.

7. Add 20% overhead with

`final executor memory = base executor memory / 0.8`.

8. Derive total cluster slots and total executor memory if needed:

`total cores = executors x cores per executor`

`total executor memory = executors x final executor memory`.

This model is intentionally simple enough for students to execute by hand. Afterward, they should validate the resulting assumptions through Spark UI observations, storage-level behavior, and cluster-runtime metrics (Apache Spark, n.d.-c, n.d.-f).

3.5.3. Five Worked Examples

For all five worked examples, the class assumptions are the following unless otherwise stated:

- Target partition size: 128 MiB
- Cores per executor: 2
- Memory amplification factor: 4
- Executor overhead: 20%
- Concurrency model: one wave unless the example says otherwise

Example 1. 10 GiB, one execution

Step 1. Convert the data size:

`10 GiB = 10 x 1024 MiB = 10,240 MiB`

Step 2. Estimate partitions:

$\text{partitions} = 10,240 / 128 = 80$

Interpretation: Spark will have 80 partition-level tasks available for this dataset.

Step 3. Estimate active tasks:

$\text{active tasks} = 80$

because the example assumes one execution and one-wave concurrency.

Step 4. Estimate executors:

$\text{executors} = 80 / 2 = 40$

Interpretation: if each executor contributes 2 task slots, 40 executors provide 80 concurrent slots.

Step 5. Estimate base executor memory:

$\text{base executor memory} = 128 \times 4 \times 2 = 1,024 \text{ MiB}$

Interpretation: each executor is assumed to keep two active partitions in flight, with a fourfold memory-expansion allowance.

Step 6. Add overhead:

$\text{final executor memory} = 1,024 / 0.8 = 1,280 \text{ MiB}$

Final classroom estimate:

- Partitions: 80
- Executors: 40
- Cores per executor: 2
- Executor memory: 1,280 MiB
- Total executor memory: $40 \times 1,280 = 51,200 \text{ MiB}$, approximately 50 GiB

Example 2. 50 GiB, one execution

Step 1. Convert the data size:

$50 \text{ GiB} = 50 \times 1024 \text{ MiB} = 51,200 \text{ MiB}$

Step 2. Estimate partitions:

$\text{partitions} = 51,200 / 128 = 400$

Step 3. Estimate active tasks:

$\text{active tasks} = 400$

Step 4. Estimate executors:

$\text{executors} = 400 / 2 = 200$

Step 5. Base executor memory remains unchanged because the partition size, amplification factor, and cores per executor are unchanged:

$\text{base executor memory} = 128 \times 4 \times 2 = 1,024 \text{ MiB}$

Step 6. Add overhead:

$\text{final executor memory} = 1,024 / 0.8 = 1,280 \text{ MiB}$

Final classroom estimate:

- Partitions: 400
- Executors: 200
- Cores per executor: 2
- Executor memory: 1,280 MiB
- Total executor memory: $200 \times 1,280 = 256,000 \text{ MiB}$, approximately 250 GiB

Interpretation: the per-executor memory did not change because the class model ties memory to partition size and local concurrency, not to total dataset size. What changed is the number of partitions and therefore the number of required task slots for one-wave execution.

Example 3. 10 GiB, two simultaneous executions

Step 1. Reuse the first dataset estimate:

- Data size: 10,240 MiB
- Partitions per execution: 80

Step 2. Because two executions must run concurrently:

$\text{active tasks} = 80 \times 2 = 160$

Step 3. Estimate executors:

$$\text{executors} = 160 / 2 = 80$$

Step 4. Base executor memory remains:

$$\text{base executor memory} = 128 \times 4 \times 2 = 1,024 \text{ MiB}$$

Step 5. Add overhead:

$$\text{final executor memory} = 1,024 / 0.8 = 1,280 \text{ MiB}$$

Final classroom estimate:

- Partitions per execution: 80
- Simultaneous executions: 2
- Active tasks: 160
- Executors: 80
- Executor memory: 1,280 MiB
- Total executor memory: $80 \times 1,280 = 102,400$ MiB, approximately 100 GiB

Interpretation: nothing changed in partition size or local executor behavior. The cluster grew because the concurrency target doubled.

Example 4. 50 GiB, two simultaneous executions

Step 1. Reuse the second dataset estimate:

- Data size: 51,200 MiB
- Partitions per execution: 400

Step 2. For two simultaneous runs:

$$\text{active tasks} = 400 \times 2 = 800$$

Step 3. Estimate executors:

$$\text{executors} = 800 / 2 = 400$$

Step 4. Base executor memory:

$$\text{base executor memory} = 128 \times 4 \times 2 = 1,024 \text{ MiB}$$

Step 5. Add overhead:

$$\text{final executor memory} = 1,024 / 0.8 = 1,280 \text{ MiB}$$

Final classroom estimate:

- Partitions per execution: 400
- Simultaneous executions: 2
- Active tasks: 800
- Executors: 400
- Executor memory: 1,280 MiB
- Total executor memory: $400 \times 1,280 = 512,000$ MiB, approximately 500 GiB

Interpretation: this result is intentionally large because the example assumes full concurrency. It helps students see how quickly resource demand grows when dataset size and concurrency increase together.

Example 5. Mixed workload: 20 GiB batch plus 20 active streaming partitions

Step 1. Convert the batch data size:

$$20 \text{ GiB} = 20 \times 1024 \text{ MiB} = 20,480 \text{ MiB}$$

Step 2. Estimate batch partitions:

$$\text{batch partitions} = 20,480 / 128 = 160$$

Step 3. Add streaming concurrency:

$$\text{active tasks} = 160 \text{ batch tasks} + 20 \text{ streaming tasks} = 180$$

Step 4. Estimate executors:

$$\text{executors} = 180 / 2 = 90$$

Step 5. Base executor memory:

$$\text{base executor memory} = 128 \times 4 \times 2 = 1,024 \text{ MiB}$$

Step 6. Add overhead:

$$\text{final executor memory} = 1,024 / 0.8 = 1,280 \text{ MiB}$$

Final classroom estimate:

- Batch partitions: 160
- Streaming partitions: 20
- Active tasks: 180
- Executors: 90
- Executor memory: 1,280 MiB
- Total executor memory: $90 \times 1,280 = 115,200$ MiB, approximately 112.5 GiB

Interpretation: the mixed workload requires the cluster to support both batch and operational flow simultaneously. In practice, architects might increase the amplification factor for stateful streaming or separate the workloads into different clusters. The classroom example keeps one shared cluster so students can see how concurrency aggregates.

These five examples are useful because they teach the mechanics of Spark sizing. They also teach a deeper lesson: the final numbers are driven by assumptions about partitions, concurrency, executor design, and working memory. Once students understand that dependency chain, they are better prepared to evaluate autoscaling, managed services, and cloud-native automation critically rather than treating them as black boxes.

3.6. Scheduling, Dynamic Allocation, and Cost Control

Spark provides both static and dynamic ways to allocate resources. Static partitioning assigns an application a fixed resource envelope for its duration. Dynamic allocation, by contrast, allows Spark to request executors when there is demand and release them when they become idle. The job scheduling guide describes this as a mechanism to adjust the resources occupied by an application according to workload changes (Apache Spark, n.d.-d).

The same documentation also makes two operational points that are frequently omitted in classroom explanations. First, dynamic allocation is disabled by default. Second, enabling it requires `spark.dynamicAllocation.enabled=true` and one of several supporting conditions, such as an external shuffle service or shuffle tracking, as documented in Spark's configuration and job-scheduling guidance (Apache Spark, n.d.-b, n.d.-d). This means autoscaling is not a magical switch; it is a coordinated runtime design choice with prerequisites.

Spark also supports scheduling policies within an application. By default, concurrent jobs run in FIFO order, but fair scheduling can distribute tasks in a round-robin manner so that shorter or high-priority workloads receive resources earlier (Apache Spark, n.d.-d). This becomes important in service-oriented or mixed-demand environments where different jobs should not wait behind one long-running batch operation.

“Business first. Data second. Cloud agnostic by design. Because in modern architectures, everything speaks the same language: Apache Spark.”

Manuel Hernandez Giuliani

From a FinOps perspective, this matters because cost optimization is not the same as indiscriminate downsizing. A good Spark FinOps posture aligns cluster behavior with workload criticality. Baseline, always-on jobs may justify stable capacity. Elastic batch jobs may justify dynamic allocation. Shuffle-heavy or cache-intensive jobs may justify additional headroom because recomputation, repeated failures, or unstable latency can be more expensive than slightly higher steady-state allocation. In academic terms, Spark cost control is a problem of workload economics, not just infrastructure arithmetic.

3.7. ManOxCo as the Exclusive Teaching Use Case

ManOxCo provides a useful way to connect Spark theory to business consequence. Consider three representative workload families. The first is daily reconciliation of oxygen production, truck dispatch, and hospital-consumption records. This is mostly a batch integration problem in which Spark can scan landed files, standardize schema, join datasets, and publish quality-checked outputs for reporting and downstream planning.

The second workload is historical dry-out risk analysis. Here the value of Spark lies in repeated transformations over larger datasets, such as combining consumption trends, route delays, replenishment history, and facility thresholds to estimate which hospitals are approaching unsafe autonomy windows. If intermediate results are reused across several analytical steps, persistence strategies become important. If grouping and aggregation by hospital, plant, or corridor are frequent, shuffle behavior becomes a core design concern rather than a minor implementation detail.

The third workload is mixed demand. A ManOxCo platform may need a near-real-time or micro-batch operational view for risk monitoring while also running heavier retrospective recalculations or planning jobs. In such a case, fair scheduling, scheduler pools, or separate

Spark applications may be better architectural choices than treating all processing as one homogeneous queue (Apache Spark, n.d.-d). The business lesson is direct: Spark architecture should reflect the life-critical asymmetry of the workload. A delay in one hospital-risk calculation does not carry the same consequence as a delay in a non-urgent historical summary.

For that reason, ManOxCo should not be used merely as a decorative example. It is a reminder that the end state of Spark design is not cluster elegance alone, but trustworthy operational decisions. In a life-critical oxygen ecosystem, Spark is valuable only insofar as it helps convert distributed data into coordinated action before a dry-out event occurs.

4. Conclusion

Apache Spark should be taught as a distributed execution architecture grounded in RDDs, partitions, actions, stages, shuffles, persistence, and explicit resource management. The official Spark documentation supports that systems-oriented interpretation far more strongly than the simplified classroom myth that Spark is just a fast engine with a few executor formulas attached to it. RDDs remain central because they make the logic of fault tolerance, laziness, and parallel execution visible.

The discussion also shows that executor and memory sizing should not be reduced to one fixed arithmetic rule. Spark's official guidance is empirical and workload-sensitive: inspect partitions, understand shuffles, sample memory behavior, reserve headroom, and validate assumptions through the monitoring UI. At the same time, students benefit from a transparent classroom method that teaches how partition counts, task slots, concurrency, executor design, and overhead interact. Finally, the ManOxCo case demonstrates why this matters. Spark architecture is worthwhile not because it can scale abstractly, but because well-designed distributed computation can support more reliable and timely decisions in a business context where operational delay may carry real human consequence.

5. Discussion Highlights

Five principal findings emerge from the discussion. First, RDDs remain the clearest conceptual entry point for understanding Spark's distributed execution model, even in environments that also use higher-level APIs. Second, Spark's runtime architecture is best understood as coordinated interaction between the driver, cluster manager, executors, and partitions rather than as a vague notion of "parallel processing." Third, jobs, stages, tasks, and

shuffles define the real operational logic of execution and explain many performance differences between apparently similar pipelines. Fourth, Spark does not endorse a universal sizing formula, but students can still learn a rigorous classroom estimation workflow if its assumptions are made explicit. Fifth, ManOxCo confirms that Spark design should always be workload- and consequence-aware, especially when platform outputs inform operational decisions in a life-critical environment.

6. Glossary

Action: An operation that triggers computation and returns a result to the driver or writes output.

Cluster manager: The resource-allocation layer through which Spark acquires executors, such as standalone mode, YARN, or Kubernetes.

Executor: A process launched for a Spark application that runs tasks and stores application data.

Lazy evaluation: The principle by which Spark delays computation until an action requires a result.

Partition: A logical slice of an RDD and the practical unit of task-level parallelism.

Persistence: The explicit decision to keep an RDD in memory, on disk, or in serialized form for reuse.

RDD: A resilient distributed dataset; a fault-tolerant distributed collection that Spark can operate on in parallel.

SparkContext: The foundational cluster-facing context through which a Spark application coordinates access to executors and cluster resources.

SparkSession: The modern application entry point that exposes Spark's structured APIs and, in classic Spark, provides access to the associated SparkContext.

Shuffle: A redistribution of data across partitions and executors, typically required for grouping or aggregation by key.

Stage: A set of tasks that can be executed before a shuffle boundary requires redistribution.

7. References

Apache Spark. (n.d.-a). *Cluster mode overview*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/cluster-overview.html>

Apache Spark. (n.d.-b). *Configuration*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/configuration.html>

Apache Spark. (n.d.-c). *Hardware provisioning*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/hardware-provisioning.html>

Apache Spark. (n.d.-d). *Job scheduling*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/job-scheduling.html>

Apache Spark. (n.d.-e). *RDD programming guide*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/rdd-programming-guide.html>

Apache Spark. (n.d.-f). *Tuning*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/tuning.html>

Apache Spark. (n.d.-g). *Getting started*. Retrieved April 18, 2026, from <https://spark.apache.org/docs/latest/sql-getting-started.html>

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). *Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing*. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* (pp. 15-28). USENIX Association. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>

Appendix A. Self-Assessment Exercises

Use the classroom assumptions unless the exercise changes them:

- Target partition size: 128 MiB
 - Cores per executor: 2
 - Memory amplification factor: 4
 - Executor overhead: 20%
1. Calculate the partitions, executors, executor memory, and total executor memory for a 20 GiB file processed in one execution.
 2. Recalculate the 50 GiB example, but use a target partition size of 256 MiB instead of 128 MiB. What changes, and why?
 3. Calculate the active tasks, executors, and total executor memory for two simultaneous executions of a 10 GiB workload.
 4. If the base executor memory estimate is 1,536 MiB, what is the final executor memory after adding 20% overhead?

5. Design the classroom estimate for a mixed workload composed of 20 GiB batch processing and 20 active streaming partitions. Explain what the combined active-task count means operationally.

Appendix B. Answer Key

1. 20 GiB = 20,480 MiB.
partitions = $20,480 / 128 = 160$.
executors = $160 / 2 = 80$.
base executor memory = $128 \times 4 \times 2 = 1,024$ MiB.
final executor memory = $1,024 / 0.8 = 1,280$ MiB.
total executor memory = $80 \times 1,280 = 102,400$ MiB, approximately 100 GiB.
2. 50 GiB = 51,200 MiB.
partitions = $51,200 / 256 = 200$.
executors = $200 / 2 = 100$.
base executor memory = $256 \times 4 \times 2 = 2,048$ MiB.
final executor memory = $2,048 / 0.8 = 2,560$ MiB.
The number of partitions falls because each partition is larger, but the memory per executor rises because each active task now carries a larger partition.
3. For one execution of 10 GiB, partitions = $10,240 / 128 = 80$.
For two simultaneous executions, active tasks = $80 \times 2 = 160$.
executors = $160 / 2 = 80$.
final executor memory = $1,024 / 0.8 = 1,280$ MiB.
total executor memory = $80 \times 1,280 = 102,400$ MiB, approximately 100 GiB.
4. final executor memory = $1,536 / 0.8 = 1,920$ MiB.
5. 20 GiB = 20,480 MiB.
batch partitions = $20,480 / 128 = 160$.
active tasks = $160 + 20 = 180$.
executors = $180 / 2 = 90$.
final executor memory = $1,024 / 0.8 = 1,280$ MiB.
total executor memory = $90 \times 1,280 = 115,200$ MiB, approximately 112.5 GiB.
Operationally, the combined active-task count means the cluster must support both the batch and streaming demand at the same time if the architect wants full concurrency.

Appendix C. Citation Mapping and Notes on the Original Source List

- Sections 3.1 and 3.4 were primarily rebuilt from the official RDD programming guide and the foundational RDD paper (Apache Spark, n.d.-e; Zaharia et al., 2012).
- Section 3.2 was aligned to the cluster runtime model described in the official cluster overview and updated to include the modern `SparkSession` entry point from the official getting-started guide (Apache Spark, n.d.-a, n.d.-g).
- Section 3.3 uses the official descriptions of transformations, actions, partitions, stages, and shuffle boundaries from the RDD programming guide and the job scheduling guide (Apache Spark, n.d.-d, n.d.-e).
- Section 3.5 replaces unsupported fixed sizing formulas with official hardware and tuning guidance plus a clearly labeled teaching heuristic, technical assumptions, and five worked examples for classroom use (Apache Spark, n.d.-c, n.d.-e, n.d.-f).
- Appendices A and B restore the quantitative exercise pattern from the earlier teaching draft, but now frame the results explicitly as pedagogical estimations rather than official Spark-prescribed values.
- Section 3.6 maps dynamic allocation, FIFO scheduling, and fair scheduling to the official job scheduling and configuration documentation (Apache Spark, n.d.-b, n.d.-d).
- Section 3.7 is an applied teaching interpretation built on the paper's Spark concepts and adapted to the ManOxCo scenario; it is an instructional synthesis rather than a claim taken verbatim from one Spark document.

Because the Spark documentation pages are living documents without a fixed publication date on the page itself, APA entries use n.d. markers plus a retrieval date. The in-text lettering (n.d.-a, n.d.-b, and so on) follows APA ordering for multiple no-date sources by the same corporate author.